in the task definition, in this case :dir and the values are the passed values from the command line.

The command-line invocation is a little weird:

```
$ rake delete_backups[code]
```

The arguments go inside square brackets, and if there are multiple arguments they are separated by commas $ rake delete_backups[code,verbose]. As an extra syntax gotcha, you can't have spaces before the brackets or on either side of the comma. So this is a little awkward.

Making it more awkward is that if you are using Z shell, this flat-out doesn't work, because Z shell uses brackets for its own purposes. You can get around this by defining an alias alias rake="noglob rake", which will turn off Z shell's use of brackets for the command. (If you don't use Z shell or this paragraph doesn't make sense, you can safely ignore it.)

So, while you can pass command-line arguments using Rake, the syntax isn't exactly easy. An alternative to consider is just using OptionParser. With OptionParser, you can pass arguments after a -- and they will get passed to the task and parsed. It's a little tricky, because you want to make sure you don't get the --, so you need to add an OptionParser call to clarify:

```
def delete(dir, pattern)
  files = Dir["#{dir}/#{pattern}"]
  rm(files, verbose: true) unless files.empty?
end

desc "Remove files whose names end with a tilde"
task :delete_unix_backups do
  dir = "."
  parser = OptionParser.new
  parser.on("-d DIR") { |opt| dir = opt }
  args = parser.order!(ARGV) {}
  parser.parse!(args)
  delete(dir, "*~")
end
```

Then you'd call this with rake delete_unix_backups -- -d code. The parse! command removes the options from ARGV, preventing Rake from considering them as their own tasks.

If a Rake task is named default, it will be executed if you invoke Rake with no parameters: just $ rake. Most Ruby applications and gems set the default task to run tests. Rake also allows you to set up two special variables called CLEAN and CLOBBER, which initially are an empty list of files that you can add to, as in CLEAN << Dir["*.bak"]. If those lists are not empty, then Rake automatically defines tasks named clean and clobber that delete all the files in the respective file list. The terms "clean" and "clobber" are inherited from Make. Typically, "clean" removes intermediate files but allows the end files to stick around, while "clobber" removes intermediate and final files.

You can find a list of the the tasks implemented by a Rakefile (or, more accurately, the tasks for which there is a description) using this:

```
$ rake -T
(in /Users/dave/BS2/titles/ruby4/Book/code/rake)
rake delete_backups          # Remove Unix and Windows backup files
rake delete_unix_backups     # Remove files whose names end with a tilde
```